one Salesforce or Zendesk link per comment. If you want to "unlink" an account from an issue or epic, simply remove the Salesforce or Zendesk link from the respective comment(s) on the issue or epic.

Supported link types:

- Zendesk Ticket: Use the link of a Zendesk ticket to connect a customer support request to an issue or epic. This will map the issue or epic to the customer's CARR (total) field value in Salesforce.
- Salesforce Opportunity (Growth): Use the link of a Salesforce opportunity when the opportunity represents net new ARR. Typically this would map to the following order types in Salesforce: 1. New - First Order, 2. New - Connected, and 3. Growth. This will map the issue or epic to the Net ARR in the linked opportunity.
- Salesforce Account (Retention): Use the link of a Salesforce account for all other use cases that don't directly correlate to net new ARR. This will map the issue or epic to the customer's CARR (total)

Edge cases worth highlighting

1. If there is no ~customer priority:: label present, a default value of 1 is assigned.
1. There can only be one ~customer priority:: label per description or comment. If there are multiple Salesforce or Zendesk links referenced in the same comment or description, we will attribute the first ~customer priority:: label to all of the links found in the comment or description.
1. If multiple comments or references on an issue link to the same account or opportunity, the most recently updated comment is used.
1. If an issue is marked as a duplicate of another issue, the GitLab team member will need to add the comment with the link and ~customer priority:: label to the respective open issue.
1. For completeness sake, /gitlab-org/... issues that are referenced in a customer's collaboration project creates an automated link between the account and the issue/epic, but the recommended path to take full advantage of the model is to add a comment directly to the /gitlab-org/... issue as ~customer priority:: can't be defined from a collaboration project.

Theme labels

You can add any number of ~theme:... labels to issues or epics to enable aggregating priority score grouped by theme labels within the dashboard. Using theme labels is a helpful for tool for segmenting different groups of related customer requested issues. Note: Aggregate values are not de-duplicated among all theme labels. For example, if the priority score of Issue A is 100 and it has two theme labels, the value aggregated for each theme label will be 100.

Priority points

You can think about a priority point representing a "vote" on an issue or epic. Each customer or prospective customer starts with a retention priority point pool and growth priority pool of 0. When a Salesforce or Zendesk link is added to an issues or epic, we default to attributing 1 "vote" to the issue to either the retention or growth priority point pool depending on which type of link was referenced.

Optionally using the ~customer priority::2 through ~customer priority::10 label in the same

comment as the Salesforce or Zendesk link provides a mechanism to cast additional votes (up to 10) on a single issue. There is no upper bound on how many retention or growth priority points a customer can "vote" in total across all issues. The fewer points, the more value each point will represent. The converse is also true. The more points that are distributed across issues, the less value each point will carry.

If you are adding a link to an issue or epic, here are some guidelines on how to think about which customer priority:: label to use:

1. Priority should be a function of the customer or prospect's ability to fully realize the value of GitLab to solve their business needs.
1. The needs of customers evolve over time, so it's a good idea to regularly update comments with existing links to reflect the changes of the priority of the given issue or epic to a customer.
1. When in doubt, ask the customer or prospect something along the lines of, "On a scale of 1-10, how important is a solution to your particular use case in terms of being able to effectively utilize GitLab to solve your immediate business needs"?

Label	Impact
~customer priority::10	Blocker: The account is likely to churn or we will not be able to win the opportunity because the customer or prospect is unable to address their business needs.
~customer priority::9	
~customer priority::8	
~customer priority::7	Critical: Not having an adequate solution to the problem is causing a significant loss of productivity or inability to achieve the desired business outcomes.
~customer priority::6	
~customer priority::5	
~customer priority::4	Major: There is currently a workaround, but it's not ideal and should be addressed in the near future.
~customer priority::3	
~customer priority::2	
~customer priority::1	Low: Purely a quality of life improvement but does not prevent the customer or prospect from realizing the value of GitLab.

Priority point value

The value of one retention or growth priority point where the value is unique to each account as it represents a distribution of the account CARR for retention or total open opportunity net ARR for growth. It is calculated by counting the total number of retention and growth priority points an account has across all issues and epics to which they are linked.

retention priority point value = account CARR / SUM(retention priority points)

growth priority point value = open opportunity net ARR / SUM(growth priority points)

Retention score

The score of an issue or epic taking into account all retention priority points and their

respective values from all accounts that are linked to the issue. This effectively represents all account CARR evenly distributed across all linked issues in a non-duplicated manner.

The value for each account is calculated on a per issue basis as follows:

account retention score = ~customer priority::1-10 x retention priority point value

Once we have the scores for each unique account that is linked to an issue or epic, we sum them to get the retention score for the issue:

retention score = SUM(account retention score, account retention score, ...)

Growth score

The score of an issue or epic taking into account all growth priority points and their respective values from all open opportunities that are linked to the issue. This effectively represents all open opportunity net ARR evenly distributed across all linked issues in a non-duplicated manner.

The value for each opportunity is calculated on a per issue basis as follows:

account growth score = ~customer priority::1-10 x growth priority point value

Once we have the scores for each unique account's opportunities that are linked to an issue or epic, we sum them to get the growth score for the issue:

growth score = SUM(account growth score, account growth score, ...)

Combined Score

This represents the combined score for growth and retention:

combined score = SUM(retention score, growth score)

Priority Score

Urgency allows us to incorporate the dimension of time into the overall prioritization model. The value ranges can be updated at any point in time depending on how GitLab leadership would like to globally influence prioritization across all product teams.

Retention urgency is a function of a customer's overall health as defined in Gainsight and their renewal date.

Overall Health	Renewal > 18 mo.	Renewal > 12 mo.	Renewal > 6 mo.
Unknown	1	1	1
Green	1	1	1
Yellow	1.5	2	2.5
Red	2	3	4

Growth urgency is a function of an opportunity close date and win probability. The value ranges were determined based on the observed distribution of probabilities and close dates across all open opportunities.

Opportunity Probability (%)	Close Date > 6 mo.	Close Date = 3-6 mo.	Close Date = 0-3 mo.
Unknown	1	1	1
61-100%	1	1	1
40-60%	1.25	1.5	2.0
0-39%	1.5	2.0	2.5

Priority score is calculated similar to how retention score, growth score, and combined score but layers in the dimension of retention and growth urgency to the model.

1. Calculate account retention scores with urgency for an issue - account retention score with urgency = account retention score x account urgency
1. Aggregate account retention scores with urgency - retention score with urgency = SUM(account retention score with urgency, account retention score with urgency, ...)
1. Calculate account growth scores with urgency for an issue - account growth score with urgency = account growth score x opportunity urgency
1. Aggregate account growth scores with urgency - growth score with urgency = SUM(account growth score with urgency, account growth score with urgency, ...)
1. Lastly, calculate the priority score by summing retention and growth urgency scores - priority score = SUM(account score with urgency, account score with urgency, ...)

Weighted Priority Score

The priority score of the issue or epic that is inclusive of effort where effort is the weight of the issue or the sum of all issues within an epic. If there is no weight defined, a Input Weight message will be shown which links to the issue or epic that needs to be estimated by the relevant product group.

weighted priority score = priority score / weight

The higher the value, the higher the return that will be realized relative to the investment required.

Practical Example

Please refer to this spreadsheet for a practical example of how the model calculates each of the computed fields. In the example, there is only a single customer, but the same methodology applies to how scores are aggregated for a given issue or epic across all customers and prospective customers.

Dashboards

Product user requested issue prioritization

View dashboard in Tableau

Column definitions:

- Notable ID: The ID of the issue or epic.
- Notable Type: If the link is on an issue or epic.
- Notable Title: The title of the issue or epic.
- Group: The relevant product group that is the DRI for the issue or epic.
- Stage: The relevant stage to which the issue or epic belongs.
- Category: The relevant category to which the issue or epic belongs.
- Notable Weight: The weight of the issue or the aggregate weight of all issues within an epic.
- Milestone: The milestone to which the issue is currently assigned or the furthest out milestone to which an issue within an epic is assigned.
- Upvote Count: The number of · an issue or epic currently has.
- Unique Accounts: The number of unique SFDC accounts linked to the issue or epic.
- Customer Reach: The number of total licenses across all SFDC accounts linked to an issue.
- Retention Score: See "Retention score"
- Growth Score: See "Growth score"
- Combined Score: See "Combined score"
- Priority Score: See "Priority score"
- Weighted Priority Score: See "Weighted priority score"

Filters:

- Notable ID
- Customer Heath Score
- Product Group
- Product Section
- Product Stage
- Salesforce Account Name
- Customer Success Manager
- Strategic Account Executive / Account Owner
- Sales Segment
- Aggregation
- Date Range

CSM user requested issue prioritization

View dashboard in Tableau

Column definitions:

- Link Last Updated Date: The date the comment or descrption containing the link on the issue or epic was last updated.
- Milestone: The milestone the issue or epic is scheduled to be completed.
- Deliverable: Whether or not the relevant engineering team has committed to delivering the issue or epic within the scheduled milestone.
- CRM Account Name: The customer or prospect name from their respective SFDC account.
- Issue Epic Title: The title of the issue or epic.
- User Request Link: Which type of link is referenced on the issue. Account and ZenDesk ticket are retention while an opportunity is growth.

Filters:

- Customer Success Manager
- Salesforce Account Name
- Strategic Account Executive / Account Owner
- Sales Segment
- Notable Type
- Notable ID
- Customer Health Score
- Product Stage
- Product Section
- Product Group
- Customer/Account

Roadmap

1. v1.1 Improvements
1. Provide snapshots of the data to better understand historical trends.
1. Explore how incorporate non-customer issues and epics such as longer horizon market opportunities and other types of work items (bugs, infradev, security, and so on) that product managers routinely need to schedule.
1. Determine the best method for pushing prioritization data back into GitLab.

Resources

- Issue Prioritization Framework Working Group
- Slack channel: wgissueprioritization
- Top-level epic
- Unfiltered (private) playlist
- Case study on the "Cost of Delay / Duration" methodology

SECTION: product/product-processes/dogfooding-for-r-d.md

Dogfood everything

The best way to understand how GitLab works is to use it for as much of your job as possible. Avoid dogfooding antipatterns
and try to find ways to leverage GitLab (instead of an external tool) whenever
possible. For example: try to use Issues instead of documents or spreadsheets
and try to capture conversation in comments instead of Slack threads.

As a PM, but also as any GitLab team member, you should actively use every feature,
or at minimum, all the features for which you are responsible. That includes
features that are not directly in GitLab's UI but require server configuration.

If you can't understand the documentation, or if you struggle to install
something, would anyone else bother to do it? This hands-on experience
is not only beneficial for understanding what the pain points are, but it also

helps you understand what can be improved and what features GitLab is missing.

When do we dogfood?

The Dogfooding process should begin during the validation phase of the Product Development flow. PMs should set up conversations with internal customers to gather their feedback on the problem and possible solutions. This helps the group by providing feedback from teams deeply familiar with GitLab, and allows internal customer requirements to be considered from the beginning. We've seen that features that have been dogfooded internally are more successful than those that have not gone through this process, and internal usage is required to achieve Viable maturity.

When a feature is minimal and moving toward viable, designated GitLab team members (and anyone else interested!) should be using the feature extensively and actively providing feedback to the PM and team developing it.

When a feature is viable and moving toward complete, designated GitLab team members should be using the feature exclusively, and actively providing direct feedback to the PM in situations where exclusive use is not possible.

Exceptions to dogfooding:

When a feature is planned and/or moving towards minimal, dogfooding is not required; although, the initial conversation to seek feedback from internal customers should occur.

GitLab the company works in a specific way, but GitLab the product is designed to work for lots of different companies and personas. When we have validated through research that the wider GitLab community wants a feature that doesn't align with how our organization works, dogfooding is not required.

For R&D Team Members

As an R&D organization, it's also our responsibility to ensure that the entire company dogfoods our product. This is aligned to understanding our customers, delivering a product that we love ourselves, and using our own product to surface improvements..

1. Add the Dogfooding and Dogfooding::Promote Feature label to the issue tracking the feature. Ensure a feedback thread has been started on the issue.
   {{% note %}}
   In the future we intend to add tracking and visibility to see what features are requesting dogfooding and the engagement associated.
   {{% /note %}}
1. Promoting features internally when they are ready for everyday use to get as many internal users as possible. This is particularly valuable when you can get relevant teams using your feature in their daily work. This can be done by recording a demo of the new functionality and sharing it with the team, running through examples of usage on Product calls, or identifying current workflows or processes the feature could help improve.

1. Including top internal user issues in the relevant category epics
when they align with our strategy.
1. Maintaining a set of internal customer DRIs who represent GitLab team members
who use GitLab for the purposes of developing and operating GitLab and GitLab.com.

Working with internal stakeholders comes with the added benefit of getting
immediate feedback that reduces cycle times and de-risks early investments in
new categories and features. Beyond feature velocity, internal dogfooding saves
the money that we would spend on third-party tools.

Internal users can help quickly identify where workflows might break down or
where potential usability or performance issues must be explored. We should
heavily weigh internal feedback to shape our perspective on customer needs,
and then compare it to what we know about both large and small customers
(and if we're unsure, then we should proactively ask).

Internal Customer DRIs

Internal Customer DRIs are the simplest way to use Reference Customers as we design and build our product categories.
We define specific DRIs in the categories.yml file.
Below are the responsibilities of an Internal Customer DRI:

1. Actively work to improve dogfooding of the features within your internal function and
provide proactive feedback to the product managers
1. Be the authoritative voice on what is required to begin dogfooding a feature within your
function, and the combined input from your function into prioritization
1. Respond to issue mentions in a timely fashion
1. Periodic meetings between DRI and product development group
1. Stay up to date on the changes in that feature area (i.e. watch the kickoff video / read the
release post)

Best practices

It's recommended to

- Identify Internal Customer DRIs early in the discovery/delivery of a new direction, and
follow the processes described there.
- Reach out to the wider team around the Internal Customer DRIs regularly to best internalize
all their insights.

SECTION: product/product-processes/early-access-program.md

Purpose

The GitLab Early Access Program aims to increase engagement with early adopters, create brand ambassadors while also increasing the level of feedback received on non-GA high-priority features.

In a secondary priority, this helps to cultivate contributions from the Wider Community in alignment with GitLab's dual-flywheel strategy & GitLab's Developer Relations strategy

Key group that formed alignment on product direction

Emilio Salvador
Justin Farris
Nick Veenhof
Steve Evangelista
Valerie Karnes

Alignments

1. There are multiple categorizations of how customers and users access features:
    1. Generally Available: These are features that are widely available to all customers and users, they may be only available via a paid subscription but otherwise won't be designed with any tag. We will offer full customer support for these features aligned with our support policy.0
    1. Experiment, Beta: These are features any user can opt in and test independent of the Early Access Program; more detail on what distinguishes Experiment & Beta is included in our Feature Support page.
    1. Early Access Program Features: PMs & Product Leadership might select features to require an opt-in to the Early Access Program. Guidance is that this feature must be behind a feature flag so it can be rolled out to a select group of interested customers.
1. The existing process of accepting the terms & conditions of our testing agreement will be followed.
1. Guidance from UX & Product should be followed how this gets implemented for a consistent user experience. We should avoid adding friction.
1. Guidance from Legal should be followed to help decide on appropriate risks.

FYI

For guidelines on how features are marked as Beta or Experimental, along with their legal and support status please refer to the following pages:
https://docs.gitlab.com/policy/developmentstagessupport/

Program nurturing activities

The following list is not complete and up for iteration. It serves as an example of activities that could be executed once we are able to communicate with our users is in place.

We aim to obtain quotes & feedback from customers that use pre-GA features in press releases, blog & release posts.
    Incentive is joint-publicity.
Through engaging and building a specific early-access community, we can re-use existing recognition & incentive mechanisms built by Developer Relations.
    Incentive is swag or donating trees when X testing activities are performed, when Y feedback items were created, leaderboards etc..
Through nurturing campaigns of active members we guide them to becoming a contributor.
    Wider Community Contribution licenses can be granted for those that want to fix bugs or

iterate on the features.

Traffic Drivers

The following list is not complete and up for iteration. It serves as an example of traffic drivers that could be executed once we are able to communicate with our users is in place.

Contributor Events such as GitLab Hackatons can be tuned to test pre-GA features and reward participants
Inclusion in our Community Office Hours
Increased social presence, blogs & social media campaigns
Customer outreach

SECTION: product/product-processes/making-gifs.md

Animated GIFs are an awesome way of showing of features that need a little more than just an image, either for marketing purposes or explaining a feature in more detail. This page holds all information on the entire process of creating a GIF.

General

> The GIF format is popular because it works everywhere and has a no-fuss UI. · Kornel

GIFs are used everywhere for a reason, but as you can read in the referenced article above, they are also expensive. Expensive in that a GIF can quickly become a big file, which takes longer to load. To create great looking GIFs that walk the line between file size and quality, some steps need be considered.

Quick and Easy

The "quick and easy" process is for your everyday usage, where performance is not that important. It's the most efficient way of creating a GIF.

Step 1

Show only what you need to. Not everything needs to be in there, its a glimpse at a single bug/feature. Keep the recorded area small and dedicated.

Step 2

Use a dedicated app that instantly outputs a GIF, see Tools - All in one.

Expert

The "expert" process is for those cases where performance is important. It takes a little longer, but can pay off for situations such as important blog posts, extremely big GIFs, and incredibly detailed GIFs.

Step 1

Show only what you need to. Not everything needs to be in there, its a glimpse at a single bug/feature. Keep the recorded area small and dedicated.

Step 2

> All my GIFs start as videos · Andy Orsow

If you want to create professional GIFs, you want to start from a video file. This can give you expert control over the output if you need it (e.g. motion blur can add additional professionalism). Video files will in most cases be created from a screen recording software, details can be found in the Tools Section

Step 3

Reduce the amount of colors visible. You can do this either be thinking beforehand what exactly you will capture or by limiting the amount of output colors exported in the resulting GIF (see options gifify). Check your result to see if it fits your needs.

Another step could be to drop duplicate frames by manually searching through all frames. For more information on this, look here. This additional step can take a lot of time. As with anything: "Only use it if you need to".

Step 4

Try to go for a minimum of 15 fps and see if the result is good enough with your "compress", "speed", and "resolution" settings.

Tools

There are many different tools to record your screen or to create GIFs. Use whichever tool you are comfortable with, but remember: "In order to create great looking GIFs that walk the line between file size and quality a certain control over each step of the expert process is preferred".

All in one

A few things are important in this section:

- Screen region support (ability to create a GIF off a small portion of the screen)
- Cursor support (ability to include your cursor in the gif)
- FPS support (ability to control the amount of frames per second of the outputted GIF, important if you want to show some interaction detail)
- Local saving of GIF (uploading to a server should only be an option)

Gifox (macOS)

Gifox is the absolute best option here, although a paid app, its reasonably priced ($14.99). It has support for all of the features, shortcuts, and then some.

Worthy of mentioning:

- Kap (free and open source!)
- Giphy capture (free and a great option!)
- Licecap (free, but limited options for output, results can have bad colors)
- ScreenToGif (Windows, free and open source with powerful editor)

FFCast + FFmpeg (Linux)

FFCast is a command line tool that wraps around ffmpeg to capture screen regions in order to record it or capture it. Optionally this could be piped into gifify.

Screen Recording

Shift-Command-5 (macOS)

On macOS Mojave and up, press Shift-Command (·)-5 to bring up controls to record the entire screen or a portion of the screen.

QuickTime (macOS)

On every Mac, QuickTime has already been installed. It features a nice screen record option and even has basic trim and splitting functions in the edit menu! Perfect for creating those video files.

Worthy of mentioning:

- Screeny (free for now)
- Gif Brewery (not free)
- ffscreencast (CLI tool, only capture the whole screen)
- CloudApp (free with a paid option)

Camstudio (Windows)

Camstudio is a free tool. Not yet tested...

FFCast + FFmpeg (Linux)

FFCast is a command line tool that wraps around ffmpeg to capture screen regions in order to record it or capture it. Optionally this could be piped into gifify.

Converting video to Gif

Gifify (CLI)

Gifify is a command line tool and gives you the most complete set of options in order to convert your video files to GIFs. It is probably the best free tool available, with the most control.

Example command:

gifify input.mov -o output.gif --resize 960:-1 --compress 0 --colors 50 --speed 1.05 --fps 15

Worthy of mentioning:

- Drop to Gif (Great free open source option to just convert on macOS with a GUI!)
- Screengif (CLI similar to gifify)
- Gif Brewery (macOS, not free)

Convert video to GIF online

- EZGif (Pretty good results and provides some settings)
- Giphy Gifmaker (You can keep your GIFs private if you have an account. Otherwise: "all of your GIF are belong to GIPHY")
- imgur Video to GIF (Create a GIF from hundreds of popular video sites. Use Download to get a GIF or link for .gifv format)

Converting screenshots to Gif

When you have a series of screenshots as png files, you can use ImageMagick to convert them to a Gif file. ImageMagick also allows to resize images.

console
convert -delay 50 -loop 0 .png output.gif

When you upload the Gif file to social media, ensure that the source image resolution is smaller than 2048x2048.

Resizing Gifs

The Gif resolution or file size may need resizing for social media uploads, or blog post integrations. Gifsicle supports resizing Gifs in one step. The following example changes the Gif width to 2000px:

console
gifsicle --resize 2000x original.gif > originalresized.gif

Relevant links

- Product Handbook

SECTION: product/product-processes/milestones.md

When planning, Product Managers plan to GitLab milestones. Here is the process for creating and maintaining them.

Product Milestone Creation

One quarter ahead, the Engineering team, in partnership with the Product team, will create all of the necessary milestones for the next quarter. Our standard practice is to have the Major release every May, resulting in:

text
XX.0 - May
XX.1 - June
XX.2 - July
XX.3 - August
XX.4 - September
XX.5 - October
XX.6 - November
XX.7 - December
XX.8 - January
XX.9 - February
XX.10 - March
XX.11 - April

Milestone start and end dates are defined as follows:

> The next milestone m+1 starts the Saturday prior to the current milestone m's release date and runs through the Friday prior to the milestone m+1's release date.

To update the milestones in GitLab, Product Operations follows these steps:

Step 1: .org

1. Ensure that the relevant milestones are created. Go to GitLab Milestones for .org
1. Click on New milestone in the top right
1. Title the milestone with the dot release that makes sense.
   - Note: We iterate through the .0 and further for each release with the .0 release every May.
1. Set the start date to be the Saturday prior to the previous releases release date
1. Set the end date to be the Friday prior to the third Thursday of the release month
1. Closing milestones happens in the Engineering workflow

Step 2: .com

1. Ensure that .com mirrors the .org milestones for consistency in Product, Marketing etc.
1. Ensure that the relevant milestones are created. Go to GitLab Milestones for .com
1. Click on New milestone in the top right
1. Title the milestone with the dot release that matches .org.
   - Note: We iterate through the .0 and further for each release with the .0 release every May.
1. Set the start date to be the Saturday prior to the previous releases release date
1. Set the end date to be the Friday prior to the third Thursday of the release month
1. Closing milestones happens in the Engineering workflow

Understanding Releases

The release definitions are maintained by the Engineering Team and we run the end of each Milestone on the release date.

Product Milestones Usage

- These milestones are used create boards and Issues for each release
- The Product Development Google Calendar (WIP) - also uses these milestone names and dates.

Relevant links

- Engineering release definitions

SECTION: product/product-processes/planning-with-gitlab.md

We use GitLab to document product strategy and manage our backlog. A couple of concepts that are key to this process are:

- Milestones: Align with our product releases and are used as our group's planning timeboxes.
- Issues: Capture an atomic piece user value.which should able to be delivered within a singe milestone.
- Tasks (optional): Decompose an Issue into more detailed implementation steps.
- Epics: Group related issues together into a theme or goal. A best practice is for epics to not be everlasting containers but to represent a concrete scope of work, with the goal is for the epic can be closed once the work is complete.
- Boards: Aid in visualizing work moving through the product development flow and for milestone planning.
- Roadmaps: Aid in visualizing epics in a timeline view.

Issues

We use issues to define narrowly scoped items of work to be done. Issues can focus on a variety of different topics: UX problems, implementation requirements, tech debt, bugs, etc. A good guideline for experience-related issues is that they should address no more than one user story. If an issue includes multiple user stories, then it is likely an epic.

When to create an issue

You should create an issue if:

- There isn't already an issue for what you're intending to write. Search first
- A feature is mentioned in chat channels like product, competition or elsewhere
- A customer requests a particular feature

You should consider not creating an issue when:

- You're breaking down work very far ahead of implementation. Create broad issues or epics for distant features to minimize the total number of issues and maintain flexibility to iterate on the plan closer to implementation.

- There is already an existing issue. Even if the quality is low, people might have linked to it. Consider rewriting the description instead and leaving a comment that you did so.

How to submit a new issue

1. Search the GitLab project to see if a similar issue already exists. We shouldn't create duplicates if we can avoid them.
1. Use an issue template to capture all the right data elements:
   - Basic proposal for minor tasks or technical details for tracking larger issues.
   - Lean feature proposal and for all feature enhancements that will get proposals and potentially become release post items.
   - Feature proposal for large new features that may require more information or details and will become release post items.
   - Bug to report undesirable or incorrect behavior.
1. Do not
    - assign someone to the issue
    - assign a milestone
    - set a due date
    - add weight - weight represents the technical complexity and should be defined by our developers
1. Assign labels for work type classification.
1. Leave a comment and tag the product manager to triage the issue.

Mentioning someone in an issue comment will trigger the notification mechanisms chosen by the people who are mentioned - therefore there is no need to notify people in another channel after the issue has been created (Slack, email).

Feature issues

Feature issues identify work to support the implementation of a feature and/or results in an improvement in the user experience.

- When considering whether an issue is a missing feature or a bug, refer to the definition of an MVC and Definition of Done for general guidance that works well in most cases.
- If there is doubt about whether you could expect something to be there or work, it's a missing feature.
- We iterate to deliver features, so we often don't have functionality that people expect. For this reason, 'people could reasonably expect this functionality' does not make it a bug.
- Whether the code results in user facing updates or not, if it is part of building the feature it should be labeled as such.
- Performance improvements and user interface enhancements improve the experience for end users and should be labeled as ~"type::feature".
- API additions including both REST and GraphQL should also be labeled as ~"type::feature".
- If people care about a missing feature and the solution is clear, the issue should be marked as ~"Seeking community contributions".
- If the missing feature has been reported by a customers, the issue should be marked as ~"customer".

Bug issues

Bug issues report undesirable or incorrect behavior, such as:

- Defects in shipped code.
- Inaccurate presentation or data.
- Part of GitLab not working according to the documentation or a universal expectation.
- Functionality inadvertently being broken, or changed from how it is supposed to work. This is also a regression.
- A security issue that is determined to be a vulnerability should be labeled as ~"type::bug" and ~"bug::vulnerability".
- Loss of data while using the product as intended or as documented. Data corruption/loss is one basis for classifying a bug as severity::1.

Epics

Issues related to the same feature should be bundled together into an
into an epic.

Epics for a single iteration

Features may include multiple issues even if we are just targeting an MVC. In
this case, we should use an epic to collect all of them in one single place.
This epic should have a start and an end date, and it should not span more than
3 releases, otherwise we run the risk of having epics drag on indefinitely.

When these issues are finished and closed, we should have successfully achieved the
epic's goal. A good example of this kind of epic is the first iteration on a new
feature. Epics representing MVCs should clearly state MVC at the end of the
title and should have a parent epic relationship towards a category strategy or a meta epic.

Epics for multiple iterations

Multi-level Epics can be used to track features that will require multiple iterations to fully implement. These epics typically don't have a specific end date, as they represent an ongoing area of development. However, they should still have a clear goal and definition of success.

Some examples of multi-iteration epics include:

- Category strategy epics that outline the long-term vision for a product area
- Epics for large, complex features that will be built incrementally over time

When creating a multi-iteration epic:

1. Clearly define the overall goal and success criteria
2. Break down the work into smaller, manageable iterations or phases
3. Create child epics or issues for each iteration
4. Regularly review and update the epic as work progresses and priorities change

It's important to balance between having a long-term vision and maintaining flexibility to adapt based on user feedback and changing priorities. Regularly reassess multi-iteration epics to ensure they still align with product strategy and user needs.

We try to avoid creating epics for time frames as reaching a certain date shouldn't be considered an exit criteria for feature development. Instead scope epics for product features with a clear user experience goal in mind.

Meta epics for longer term items

To categorize issues and epics into themes or longer running items, we recommend using labels. You can also use epics to track many issues related to a specific topic, even if there isn't a specific timeline for shipping. These epics should be marked as ~meta, they may not have a specific start or end date, and may contain single iteration epics. This approach can be problematic due to epics and issues within a meta epic may need to be re-parented to represent a work breakdown structure and then lose the meta epic relationship.

Work item state

When an issue or epic is open, this signals that we're intending or considering implementing that change.
It's important to close items that we're not considering implementing in the near future, so that we avoid creating uncertainty with customers and colleagues.

When to close an issue or epic

To clearly communicate to our stakeholders what we are going to do, it's critical that you not only
provide the positive view (what we will do), but also articulate the negative view (what we will not
do). While this should be communicated in stage and category strategies,
it starts with your work items (issues, epics and tasks). Rejecting a feature request or a merge request is not an easy thing. People can feel quite protective of their ideas. They might have invested a lot of time and energy in writing those ideas. You can be tempted to accept a feature only to avoid hurting the people who thought of it. Even Worse, if you reject an idea too harshly, you might discourage other people to contribute, which is something we should strive to avoid.

As a Product Manager you should close work items for the following reasons:

1. Duplicate Issue: GitLab has a large issue tracker and it can be challenging for users to find other issues that might be the same request or bug. PMs should make sure they're triaging their issue lists regularly to close duplicate issues so that it's easier for users to find the issue they're looking for and for PMs to understand the demand for requests. You should use the /duplicate quick action to close these and link them to the canonical issue.
1. Is outside the scope of, or is opposed to, our vision. GitLab is a large platform and not all requests will align with the long term direction of the product. It's okay to close issues that we'll never do because they do not align with this vision.
However, you should not close an issue just because it isn't currently prioritized as part of your category's direction.
1. It presents a security risk. Some requests may require you to evaluate if the proposal can be delivered without presenting a security risk to GitLab or our customers. You should reach out to Application Security if you are unsure about any requests.
1. Too complex: We want to have a simple, user-friendly product that does complex things, not

the other way around. Uses of the product that are overly complex or only solve for narrow use cases might fall into this category.
1. We don't want another setting: whenever we can, we try to avoid having settings. GitLab follows the Convention over Configuration principle when evaluating new proposals. It's important to consider all of the user experience impacts an additional setting can add and while some settings are unavoidable; most aren't.
1. Changes the tier of a feature: this problem is already addressed in the Stewardship page.
1. No longer relevant: You should close issues where the feature may have already been delivered through some other solution or a bug may have been resolved or eliminated through a different effort.
1. Deprecated, removed, or no-longer-supported functionality: You should close issues that won't be worked on because the functionality has been officially deprecated or removed, or has reached End of Support.
Closing work items whenever possible is an important part of your job and helps to keep a clear view of what is next.

When closing a work item, leave a comment explaining why you're closing it, and link to anything of relevance (the other duplicate, the original feature that this is an iteration on, etc). Don't forget to thank the authors for the time and effort taken to submit the feature request/merge request. In all cases and without exception, you should be nice and polite when interacting with users and customers.

When you should NOT close an issue or epic

1. Low priority: sometimes features are interesting but we simply don't have the capacity to implement them. In that case, simply tell the truth and indicate that we don't have enough resources at our disposal to do it at the moment. Use the ~low priority label to signal low priority.
1. Not part of your direction: These items are good ideas, but are not at the top of the list for PMs to prioritize within their group. Use the ~low priority label to signal low priority.
1. Low demand for the request: Something that is in line with your direction but very low priority with no (or few) upvotes. Rather than closing, utilize the %"Awaiting further demand" milestone.
1. Divested category: Issues for categories in which we've made a divestment, but haven't removed the category. Use the ~divested label to signal no prioritization due to divestment.
1. Age of Issue: Closing due to age alone. Filtering by age to look for candidates to close is fine, but if the issue still aligns with product direction and there is community interest, we should keep these open for future opportunities.
1. Complex Solutions: Sometimes issues may come with overly complex proposals or the current state of GitLab's architecture or other technical factor makes the solution too complex to implement. These issues should remain open if the problem to solve is valid as solutions can evolve as more is learned about the problem to be solved.

Roadmaps

A roadmap
for your group can aid in tracking timeline oriented long-running efforts (here's an example). This can help keep work well organized, track progress and surface dependencies.

Boards

As part of our planning process it is important that you maintain a prioritized issue board for your group.
It's customary to call these boards STAGE - GROUP - Planning and to configure them to filter to all issues with your group label and with each milestone as a column (here's an example).

As the DRI for milestone prioritization it is your responsibility to ensure that all items on your Planning board are scheduled to a milestonemilestones and are prioritized both within and across milestones. This means the lowest priority in the current milestone would generally be the top priority in the next milestone.
In this regard your planning exercise is a complete prioritization of the near term issues.

SECTION: product/product-processes/product-launch.md

{{% include "includes/product-handbook-links.md" %}}

Context

Launching a product or service at GitLab involves coordination across a wide range of teams. Launching products smoothly is a critical part of delivering value to the customer, ensuring a positive customer experience, and positioning us to meet our financial targets. Our ability to efficiently manage the New Product Introduction (NPI) process will become increasingly important as we bring new products to market more frequently. To ensure that we launch new product offerings, service offerings, or SKUs effectively; we require a set of steps that happen in phases.

You do not need to follow the NPI process to launch features into our existing product set and tiers. A feature that will fit into our existing monetization options (free, premium, ultimate) follows our normal operating procedure of launching into a release milestone.

Summary of phases

1. NPI planning: teams & a Steering Committee (SteerCo) define scope and requirements; agree on prioritization of NPI project against other priorities
1. Project Execution and Launch: teams work based on their own plans, with close coordination on decisions and steps that impact others
1. Feedback: contributing teams retro and evaluate how the NPI process could be improved in future iterations

Criteria for using this process

What makes a product, service, or SKU appropriate for the NPI process?

1. It constitutes a new product or service (i.e. it is not just functionality that cleanly slots into existing product tiers or existing SKUs)
1. It is as operationally complex as a new product, service, or SKU; even if not new. (e.g. new packaging, business model, or SKU that requires custom systems configuration or revenue treatment)
1. It represents a significant enough financial or strategic opportunity to warrant investment

of dozens of teams' effort
1. The process may also be applicable if a product, service, or SKU is associated with a significant external announcement or event and we want to ensure operational excellence

How do we determine whether it's ready to start the NPI process?

In order to launch the NPI process, the DRI must communicate:

1. What we're launching: product features & functionality, service entitlements, or something else
1. Why we're launching: value proposition, strategic value, and scale of financial opportunity (understand that finance will not have built detailed models, but some financial justification must exist)
1. Who we're targeting: Ideal Customer Profile (ICP) and target market
1. When we need to launch: any internal (e.g. Field blackout periods, month/quarter/year end financial close, quiet periods, etc.) or external factors that create timing requirements
1. That we're able to launch: product development is sufficiently far along that it makes sense to start the process, and GTM teams have sufficient bandwidth to do the work in the required timeframe

What happens if a product or service doesn't meet the criteria?

Possible routes:

1. The SteerCo needs more information · sends the proposal back to the DRI
1. The SteerCo agrees the product/service is suitable, but teams do not have capacity · problem-solve with DRI & E-Group (prioritize & sequence launches, reallocate resources and/or adjust launch scope)
1. The SteerCo doesn't think the product/service is suitable for the NPI process · discuss with DRI & E-Group

A Note on NPI Process Updates

We are actively iterating on the NPI process. You can use the npi-process-working-group Slack channel to follow along about the work as well as to share any feedback you may have. Since this is an active change to ongoing processes, we've left the original process instructions at the bottom of this page. This section will be removed once the revised NPI process is finalized.

Phase 1: NPI Planning

This phase happens before the NPI process officially launches with cross-functional stakeholders.

Step 1: Write up the details of the product or service

The DRI should create a business plan based on an NPI Planning Info Sheet (template not finalized, to be linked once done) to communicate how you've defined the specific product or service that is being launched. At this point in the project you may not have all the details - use this resource as a starting point to help you and your stakeholders understand what still

needs to be resolved cross-functionally over the course of the project.

Ideally at this point you will already know the pricing structure, but if not, a conversation with the Pricing Committee is needed:

1. Contact Justin Farris's EBA to schedule time with the Pricing Committee (currently Sean Hall and Justin Farris)
1. Pricing will review the product or services brief and follow up within a week for any clarifying questions, and suggest the right level of research needed to arrive at a recommendation.
1. Pricing team will conduct required research following our process, and pricing principles. Depending on the complexity of the offer, and potential market size this research could take a couple of weeks, up to a few months to complete.
1. Pricing team will make a recommendation of what business model the product/service should have (e.g. fixed fee, subscription, consumptive, add-on, new tier), and recommend a price point or range to consider for the offering

Step 2: Meet with the NPI SteerCo and get alignment on the launch plan

The DRI/Business Sponsor and the Program Manager meet with the NPI SteerCo and present the information included in the NPI Planning Info Sheet.

The NPI SteerCo will advise on whether there are available resources to complete the NPI process for your proposed feature or service. In some cases, there may not be sufficient resources to match the prioritization level of a given project. In that case, it is important for the Business Sponsor and the NPI SteerCo to work together to escalate to the E-Group for further direction.

To meet with the SteerCo, reach out to Natalie Pinto or Lee Work who will add you to the next meeting.

Step 3: Assign a DRI and Program Manager to launch the product/service or SKU

Once approved by the NPI SteerCO, the project should have both a DRI and a Program Manager. The DRI drives the feature or service strategy and is the primary business owner. This is the person requesting a new product, service, or SKU. The person is responsible for delivering the initial business plan, and ensuring that the launch meets business goals.

The Program Manager coordinates and directs the multiple parallel work streams required by the NPI process. The Program Manager can be assigned from the Product division, or if necessary from the Office of the CEO. The Professional Services team can Program Manage SKU additions or modifications for Professional Services. In some cases, the DRI and the Program Manager may be the same person.

Step 4: Communicate the plan to stakeholders

Once the NPI SteerCo has given your project the go-ahead, it's time to get the rest of the stakeholders aligned. Reminder: there are many steps required to deliver a net new SKU; this isn't a process for processes-sake. We've learned through multiple NPI projects that this kind of work can't be completed efficiently without clear communication and transparency in the

process (NPI's might be limited access in which case, details would not be open to all team members only those that need-to-know).

The Program Manager should begin coordinating efforts and identifying the relevant stakeholders across the organization. We recommend hosting the following meetings as soon as possible in order to kickoff the project:

1. Stakeholder kickoff (process-focused): Meeting with all NPI stakeholders to launch the process: share overview of timeline, milestones, what success looks like, expected challenges.
2. Product review (product-focused): Product Manager to present overview to all stakeholder teams. This is a deeper dive than stakeholder kickoff, and more focused on the product. GTM teams use this information for additional due diligence & clarity, and as an input to start developing their own strategies and roadmaps. (e.g. Deal Desk may have questions around usage boundaries, approval processes for non-standard steps, etc.)

These meetings are a chance for the various stakeholders and Subject Matter Experts (SMEs) in the organization to raise risks/issues sooner rather than later. The SME may ask questions that you can't answer yet; that's an opportunity for further discussion later in the project. For stakeholders this is a chance to understand what will be expected of them in the coming weeks/months.

We acknowledge that synchronous meetings aren't GitLab's ideal way of working, however in cases of complex cross-functional projects it can be challenging to make decisions and keep teams aligned without synchronous check-points. Our suggestion is to approach these meetings as "advised, but optional." The project leads can use their own judgement to decide if a given meeting is necessary for their project and whether it can be optional for some members of the larger team.

Step 5: Organize logistics

The Program Manager should ensure that the team is set up for success. The following resources are valuable, but not mandatory. We recommend creating a Single Source of Truth (SSOT) with a link out to the following resources/information:

1. Project epic (or group/project, whatever makes sense for your project)
1. Slack channels
1. Recurring meetings (on whatever cadence makes sense for your project)
1. Plan for recurring async updates
1. Define project milestones/timeline
1. Project planner

Phase 2: Project Execution and Launch

This phase of the NPI project is generally handled by the stakeholder teams. They are subject matter experts in their own tasks / needs and can be trusted to manage their work accordingly. These stakeholders will create the relevant artifacts necessary for their function. For example: GTM plans, marketing message and vision, partner plans, enablement materials, sales projections, gross margin projections including hosting/infrastructure costs, support/PS costs, etc.

The Program Manager and DRI should be continually working with teams to ensure that work is on track against the defined timeline. The Program Manager should also partner with teams to facilitate decision making and problem solving where needed. NPI projects can be highly complex and the teams will need to discuss challenges regularly.

An important note: occasionally the team may encounter decisions that dramatically change the parameters of the project. In this case, we ask that the Program Manager and DRI realistically approach the long-term plan. If the stakeholders say they cannot complete the project on time given the changes, then the plan needs to be adjusted.

In the future this section will include detailed project plan templates for teams to use and collaborate on together - this will be more action focused for the broader team.

Phase 3: Feedback

Once the new feature or service has launched, the Business Owner and Program Manager should organize a retrospective. This will inform future NPI projects so we can continue to iterate and improve.

Teams should also follow up post-launch with input on customer perspective/feedback, as needed.

{{% details summary="Click here to expand and see the earlier version of this process."%}}

Context

Launching features into our current product set is a well understood process. To launch new product offerings, service offerings or SKUs requires a set of steps that need to be sequenced in a way to ensure product launch is done efficiently and effectively.

Summary of the steps.

1. Business Sponsor and a program manager should be assigned to launch the product/service or SKU.
1. The product or service needs to be clearly defined
1. Pricing research / pricing and packaging plan should be created
1. An operations / business plan / financial projection should be developed for the product / service
1. Approvals should be obtained to move forward with the launch
1. Product Launch checklist should be created with clear DRIs assigned
1. Execution against Product Launch checklist should happen

Step 1) Business Sponsor / program manager should be assigned to launch the product/service or SKU.

A Business Sponsor should request a new product, service, or SKU. The business sponsor can be the DRI to drive the SKU launch if it is smaller in scope. If the scope is larger, a program manager will be assigned to execute, and launch the product, service or SKU. A program manager can be assigned from the Product division, or if necessary from the Office of the CEO. SKU

additions or modifications for professional services can be program managed by the Professional services team.

To initiate the work (and proceed to steps 2 to 7) provide a three sentence proposal to the Pricing Steering Committee to kick off steps 2 to 4. The committee meets monthly and will review and provide any necessary feedback, and ensure the team is aligned on supporting the follow up work.

Step 2) The product or service needs to be clearly defined

A services brief should be drafted. This should include:

1. Details of the service offering (e.g. 24x7 support, emergency support SLAs, PS SOW description, etc)
1. Description of the market and ideal customer
1. What the customer gets in terms of entitlement
1. What is required of GitLab to deliver the service
1. Hypotheses of how the product will be priced (e.g. Hourly, fixed fee, subscription)

A product brief: (use this template)

1. Description of the product.
1. Description of the market and ideal customer
1. What features are available today that will be part of the product.
1. What features will be part of the offering
1. If the product will be included as part of our primary tiers or as an add-on
1. Hypotheses of how the product will be priced (e.g. new tier, add-on, consumption)

Step 3) Pricing research / pricing and packaging plan should be created

- Pricing will review the product or services brief and follow up within a week for any clarifying questions and suggest the right level of research needed to arrive at a recommendation.
- Pricing team will conduct required research following our process, and pricing principles. Depending on the complexity of the offer, and potential market size this research could take a couple of weeks, up to a few months to complete.
- Pricing team will make a recommendation of what business model the product/service should have (e.g. fixed fee, subscription, consumptive, add-on, new tier), and a recommended price point or range to consider for the offering

Step 4) A Operations/ business plan / financial projection should be developed for the product / service

For a new product or service we should build an operational business plan (this this template). This includes

- Product plan
- Marketing message and vision
- Ideal customer profile
- GTM Plan

- Partner Plan
- If necessary a infrastructure cost/operating model for the offering

For a new services offering we should also include a:

- Staffing Plan
- Operational Model
- Systems changes required to support the new service

Build a financial model

- Topline sales projections
- Gross Margin projections including hosting/infrastructure costs, support costs, professional services costs, etc
- Possibly model contribution including other costs such as Sales and Marketing or R&D.
- Any financial asks needed to support the product (e.g. support staffing)

Step 5) Approvals should be obtained to move forward with the launch

With the product/services proposal, pricing research, operational plan and business plan in place, the leadership can make a decision to move forward. This approval with the correct structure will accelerate the next phase which is planning and execution.
Approvals will follow this matrix.

Once the approval is made the following teams need to be notified that a workstream is kicked off:

- Product offering: Product owners and engineers
- Services offering: Professional services or support team
- All offerings: Product Marketing, Legal, Product Fulfillment, IT, Data, FP&A, Billing, Revenue accounting, Tax

Step 6) Product Launch checklist should be created with clear DRIs assigned

- Example Template is here.
- The program manager should fill out the product launch checklist and ensure there are owners for the generic items but also adds custom items.

The following areas may need to be considered in the planning:

1. Marketing Plan including positioning, press, customer references, etc
1. Product Plan including Beta programs, code complete, launch criteria, data logging, etc
1. Sales readiness including GTM plan and enablement of field and channel
1. Pricing and Packaging
1. Legal readiness including terms of service, data retention, privacy or product counsel issues
1. Customer Support / Success readiness
1. Product fulfillment including licensing, upgrade, downgrade, trial
1. Billing including preparation to sell on all sales channels (sales assisted, web direct, channel)

1. SKU creation including Zuora Billing, Zuora Revenue, CustomerDot, SFDC, Netsuite readiness
1. Revenue readiness including SSP impact and rev rec rules
1. Tax readiness
1. Data including operational metrics, impact to reporting, data warehouse ready
   - If cross functional data pulls/analysis will be required, please identify Data DRIs that can help. As an example, please see what was done for the Storage Limits Initiative
1. Metrics and comp including impact to NetARR and compensation
1. Finance including financial forecast
1. Investor Relations including investor messaging

Step 7) Execution against Product Launch checklist should happen

Execute against the plan created above some resources that are helpful are:

- SKU Creation handbook page
- Premium Pricing Execution (Team member access only)

{{% /details %}}

SECTION: product/product-processes/product-safe-guidance.md

Overview

This guide for GitLab Product Managers clarifies and expands on the Regulation FD Training.

Making changes to this page

To make any edits to this page, please create a merge request and add a description of what you want to change and why. Add labels product operations prodops:release and product handbook. Add Product Operations DRI/Maintainer @fseifoddini as Reviewer for collaboration and approval. If Product Operations is unavailable and the topic is time-sensitive, please add Maintainer @gweaver for collaboration and approval.

Please note the content of this page needs to remain aligned with Legal team guidance so any changes must be approved by one of the Maintainers.

General Guidelines

GitLab Legal has put together a comprehensive framework to help team members determine which information is internal only and that which can be shared publicly. Following the SAFE framework will help you comply with the requirements of Regulation Fair Disclosure.

If you're not sure

Please reach out to Slack SAFE with any questions that are still unclear after reviewing this page. When posting in SAFE tag Product Operations, as that will help them maintain this page. We also encourage you to raise an MR to update the handbook as needed based on your findings.

Required Disclaimers

The following applies to GitLab artifacts that have product and specific feature information. Please remember these artifacts should only contain information that is SAFE. Links to various disclaimers are available in the "helpful legal references" section on this page.

Topic	Disclaimer	Legal Review Required (Y/N)	Other Considerations
3 year direction videos	Y	Y	
Product demos, walk-through videos	N	N	
Meeting recordings (e.g. Team calls, PM Weekly, Retrospectives)	N	N	
Direction pages	Y	N	
General product handbook pages	N	N	
Epics (non-direction)	N	N	
Issues (non-direction)	N	N	
Epics ~direction	Y	N	
Issues ~direction	Y	N	
Merge Requests	N	N	

General guidance on topics/actions

Topic	Legal Considerations	Legal Review Required (Y/N)
Acquisitions	Y	Y
Pricing Changes	Y	Y
New product launches (e.g GL Dedicated)	Y	Y
References to revenue	Y	Y
User confidentiality	Y	Y

Materially Non-Public Information (MNPI)

Product managers often need access to MNPI to do their job. As GitLab is now a publicly-traded company, it is important we all understand what MNPI is so we manage information/data appropriately. Here are some examples of MNPI:

- historical or forecasted revenues, earnings or other financial results;
- significant new products or services or other product developments;
- significant new contracts or partners or the loss of a significant contract or partner;
- significant developments regarding GitLab's technology or business operations;
- possible mergers or acquisitions or disposition